

Manejo de eventos

Escuchar eventos

Podemos usar la directiva `v-on` para escuchar eventos DOM y ejecutar algunos JavaScript cuando se activan.

Por ejemplo:

```
<div id="example-1">
  <button v-on:click="counter += 1">Add 1</button>
  <p>Se ha hecho clic en el botón de arriba {{ counter }} veces.</p>
</div>

var example1 = new Vue({
  el: '#example-1',
  data: {
    counter: 0
  }
})
```

Resultado:

Métodos Manejadores de eventos

Sin embargo, la lógica para muchos controladores de eventos será más compleja, por lo que no es posible mantener su JavaScript en el valor del atributo `v-on`. Es por eso que `v-on` también puede aceptar el nombre de un método al que te gustaría llamar.

Por ejemplo:

```
<div id="example-2">
  <!-- `saludar` es el nombre de un método definido a continuación -->
  <button v-on:click="saludar">Saludar</button>
</div>

var example2 = new Vue({
  el: '#example-2',
  data: {
```

```

    name: 'Vue.js'
  },
  // definir métodos bajo el objeto `methods`
  methods: {
    saludar: function (event) {
      // `this` dentro de los métodos apunta a la instancia de Vue
      alert('Hola ' + this.name + '!')
      // `evento` es el evento DOM nativo
      if (event) {
        alert(event.target.tagName)
      }
    }
  }
})

// también puedes invocar métodos en JavaScript.
example2.greet() // => 'Hello Vue.js!'

```

Resultado:

Métodos Manejadores en línea

En lugar de enlazar directamente con un nombre de método, también podemos usar métodos en una declaración de JavaScript en línea:

```

<div id="example-3">
  <button v-on:click="di('hola')">Di hola</button>
  <button v-on:click="di('que')">Di que</button>
</div>

new Vue({
  el: '#example-3',
  methods: {
    di: function (mensaje) {
      alert(mensaje)
    }
  }
})

```

Resultado:

A veces también necesitamos acceder al evento DOM original en un controlador de instrucciones en línea. Puedes pasarlo a un método usando la variable especial `$event`:

```
<button v-on:click="Alertar('El Form no se puede enviar aun.', $event)">
  Enviar
</button>

// ...

methods: {
  Alertar: function (message, event) {
    // ahora tenemos acceso al evento nativo.
    if (event) event.preventDefault()
    alert(message)
  }
}
```

Modificadores de eventos

Es una necesidad muy común llamar

a `event.preventDefault()` o `event.stopPropagation()` dentro de los controladores de eventos. Aunque podemos hacer esto fácilmente dentro de los métodos, sería mejor si los métodos fueran puramente sobre lógica de datos en lugar de tener que lidiar con los detalles del evento DOM.

Para solucionar este problema, Vue proporciona **modificadores de eventos** para `v-on`. Recuerde que los modificadores son directivas postfijos marcados por un punto.

- `.stop`
- `.prevent`
- `.capture`
- `.self`
- `.once`
- `.passive`

```
<!-- Se detendrá la propagación del evento click. -->
```

```
<a v-on:click.stop="hazEsto"></a>
```

```
<!-- El evento de enviar ya no volverá a cargar la página. -->
```

```
<form v-on:submit.prevent="onSubmit"></form>
```

```
<!-- Los modificadores pueden encadenarse -->
```

```
<a v-on:click.stop.prevent="hazEsto"></a>
```

```
<!-- solo el modificador -->
```

```
<form v-on:submit.prevent></form>
```

```
<!-- utilizar el modo de captura al agregar el detector de eventos -->
```

```
<!-- es decir, un evento dirigido a un elemento interno se maneja aquí antes de ser manejado -->
```

```
<div v-on:click.capture="hazEsto">...</div>
```

```
<!-- solo activa el controlador si event.target es el elemento en sí -->
```

```
<!-- es decir, no de un elemento hijo -->
```

```
<div v-on:click.self="hazEso">...</div>
```

El orden es importante cuando se usan modificadores porque el código relevante se genera en el mismo orden. Por lo tanto, el uso de `v-on:click.prevent.self` evitará **todos los clics** mientras que `v-on:click.self.prevent` solo evitará clics en el elemento en sí.

Nuevo en 2.1.4+

```
<!-- El evento de clic se activará como máximo una vez. -->
```

```
<a v-on:click.once="hazEsto"></a>
```

A diferencia de los otros modificadores, que son exclusivos de los eventos DOM nativos, el modificador `.once` también se puede usar en [eventos personalizados](#). Si aún no ha leído sobre componentes, no se preocupe de esto por ahora.

Nuevo in 2.3.0+

Vue también ofrece el modificador `.passive`, correspondiente a [la opción pasiva de `addEventListener`](#).

```
<!-- El comportamiento predeterminado del evento scroll (scrolling) sucederá -->
```

```
<!-- inmediatamente, en lugar de esperar a que se complete `onScroll` -->
```

```
<!-- en caso de que contenga `event.preventDefault ()` -->
```

```
<div v-on:scroll.passive="onScroll">...</div>
```

El modificador `.passive` es especialmente útil para mejorar el rendimiento en dispositivos móviles.

No use `.passive` y `.prevent` juntos, ya que `.prevent` se ignorará y su navegador probablemente le mostrará una advertencia. Recuerde, `.passive` comunica al navegador que no desea evitar el comportamiento predeterminado del evento.

Modificadores de Teclas

Cuando escuchamos eventos de teclado, a menudo necesitamos verificar códigos de teclas comunes. Vue también permite agregar modificadores clave para `v-on` cuando se escuchan eventos de teclado:

```
<!-- solo llame a `vm.submit ()` cuando el `keyCode` es 13 -->
```

```
<input v-on:keyup.13="submit">
```

Recordar todos los `keyCodes` es un dolor de cabeza, por lo que Vue proporciona alias para las teclas más utilizadas:

```
<!-- lo mismo que arriba -->
```

```
<input v-on:keyup.enter="submit">
```

```
<!-- También funciona como abreviacion. -->
```

```
<input @keyup.enter="submit">
```

Aquí está la lista completa de alias:

- `.enter`
- `.tab`
- `.delete` (captura ambas teclas “Delete” y “Backspace”)
- `.esc`
- `.space`
- `.up`
- `.down`
- `.left`
- `.right`

También puede **definir alias modificadores de tecla personalizados** a través del objeto global `config.keyCodes`:

```
// habilita `v-on:keyup.f1`
```

```
Vue.config.keyCodes.f1 = 112
```

Modificadores automáticos de teclas

Nuevo in 2.5.0+

También puede usar directamente cualquier nombre de tecla válido expuesto a través de `KeyboardEvent.key` como modificadores convirtiéndolos a kebab-case:

```
<input @keyup.page-down="onPageDown">
```

En el ejemplo anterior, solo se llamará al controlador si `$event.key === 'PageDown'`

Algunas teclas (`.esc` y todas las teclas de flecha) tienen valores `key` incoherentes en IE9; se deben preferir sus alias integrados si necesita compatibilidad con IE9.

Modificador de Teclas del Sistema

Nuevo en 2.1.0+

Puede usar los siguientes modificadores para activar eventos listeners de raton o teclado solo cuando se presiona la tecla modificadora correspondiente:

- `.ctrl`
- `.alt`
- `.shift`
- `.meta`

Nota: En los teclados de Macintosh, meta es la tecla de comando (⌘). En los teclados de Windows, meta es la tecla de Windows (⊞). En los teclados de Sun Microsystems, el meta está marcado como un diamante sólido (◆). En ciertos teclados, específicamente los teclados y sucesores de máquinas MIT y Lisp, como el teclado Knight, el teclado space-cadet, el meta está etiquetado como “META”. En los teclados de Symbolics, el meta está etiquetado como “META” o “Meta”.

Por ejemplo:

```
<!-- Alt + C -->
```

```
<input @keyup.alt.67="clear">
```

```
<!-- Ctrl + Click -->
```

```
<div @click.ctrl="doSomething">Do something</div>
```

Tenga en cuenta que las teclas modificadoras son diferentes de las teclas normales y cuando se usan con eventos `keyup`, deben presionarse cuando se emite el evento. En otras

palabras, `keyup.ctrl` solo se activará si sueltas una tecla mientras mantienes presionada la tecla `ctrl`. No se activará si suelta la tecla `ctrl` solo. Si desea tal comportamiento, use el código de tecla para `ctrl` en su lugar: `keyup.17`.
Modificador `.exact`

Nuevo en 2.5.0+

El modificador `.exact` permite el control de la combinación exacta de modificadores del sistema necesarios para desencadenar un evento.

```
<!-- esto se disparará incluso si se presiona Alt o Shift -->
```

```
<button @click.ctrl="onClick">A</button>
```

```
<!-- esto solo se activará cuando se presione Ctrl y no se presionen otras teclas -->
```

```
<button @click.ctrl.exact="onCtrlClick">A</button>
```

```
<!-- esto solo se activará cuando no se presionen modificadores del sistema -->
```

```
<button @click.exact="onClick">A</button>
```

Modificador de Boton del Raton

Nuevo en 2.2.0+

- `.left`
- `.right`
- `.middle`

Estos modificadores restringen el controlador a eventos activados por un botón específico del raton.

¿Por qué Listeners en HTML?

Es posible que le preocupe que todo este enfoque de escucha de eventos viole las viejas buenas reglas sobre la “separation of concerns” (“separación de preocupaciones”). Tenga la tranquilidad de que todas las funciones y expresiones del controlador de Vue están estrictamente vinculadas a ViewModel que esta manejando la vista actual, no causará ninguna dificultad de mantenimiento. De hecho, hay varios beneficios en el uso de `v-on`:

1. Es más fácil ubicar las implementaciones de la función de controlador dentro de su código JS al ojear la plantilla HTML.
2. Una vez que no tenga que adjuntar manualmente eventos listeners en JS, su código de ViewModel puede ser de lógica pura y libre de DOM. Esto hace que sea más fácil de probar.
3. Cuando se destruye un ViewModel, todos los eventos listeners se eliminan automáticamente. No tiene que preocuparse por limpiarlo usted mismo.